
jerrypi

把 Pi coding agent 装进 VS Code 侧边栏 —— 不装 Node、不跑脚本，装完扩展就能用

What's in here

- 01 为什么做 约束逼出来的设计
- 02 Pi 的优势 简单 · 可控 · 好懂 · 可扩展
- 03 经验教训 语义陷阱与断言方法
- 04 推荐尝试 五分钟能上手的四件事

One extension is the whole setup

起因是一台什么都不许装的 Windows 工作机 —— 但省事的不止这一台。

- 01 **不装环境** 目标机不用装 Node —— 扩展宿主自带 Node 24
- 02 **开箱即用** pi 的 SDK bundle 随扩展分发，面板与工具卡片装完就有
- 03 **一份配置** 与 pi CLI 共享 ~/.pi/agent，会话两边互通

唯一被允许的安装方式，逼出了最省事的安装方式。

Why Pi: simple, controllable, hackable

A 简单

一个 npm 包、零原生依赖；配置就是 `~/.pi/agent` 下几个 JSON 文件。

B 可控

模型、工具、审批门槛全是自己的设置；会话是纯 JSONL，可审计可迁移。

C 好懂

文档与源码齐全，行为可预测 —— 出问题能顺着查到根因。

D 可扩展

TypeScript 扩展、SDK、主题；这个 VS Code 扩展本身就是用 SDK 拼出来的。

能用 SDK 拼出一个 IDE 扩展，就是可扩展性最好的证明。

Same names, silent traps

- 01 **名同义异** `sessionDir` 是直接放 `.jsonl` 的那层目录 —— 差一层就没影
- 02 **路径分叉** 物理路径与逻辑路径没对齐，同一个目录变成两个
- 03 **越权删除** `logout()` 会删掉用户 `auth.json` 里自己的凭据
- 04 **静默跳过** 无凭据时刷新模型列表，一次请求都不发

这些代价全是静默的 —— 所以只读源码，不猜语义。

Assertions that can go red

- 01 每步先写断言、看红、再实现 —— 编译不过、模块不存在，都不算红。
- 02 断言要独立于被测实现：隔离目录起测，或直接问另一份实现当 oracle。
- 03 **SKIP 不是 PASS** —— 静默跳过必须和「真的没有」区分得开。

每次改动自动跑 329 条 host-check 与 16 项 gating self-test；验收挂了就再来一轮。

假绿比红贵 —— 它让你以为已经验证过了。

Four ways to poke at it

- 01 **装扩展**
VSIX 仓库里的 jerrypi-0.1.12.vsix —— Install from VSIX
- 02 **试 CLI**
TERMINAL 有终端的机器直接跑 pi，会话与配置两边互通
- 03 **写扩展**
EXTEND 抄 pi-config 的 footer 扩展，几十行加一条命令
- 04 **换模型**
MODELS models.json 加自定义 provider，两边同时生效

Thanks

- 01 预发布 **0.1.12** 已可用，反馈直接找我。
- 02 感谢帮忙做真机验收的同事们。
- 03 下一步是 **S9** 与 **0.2.0** 正式版。

感谢支持。